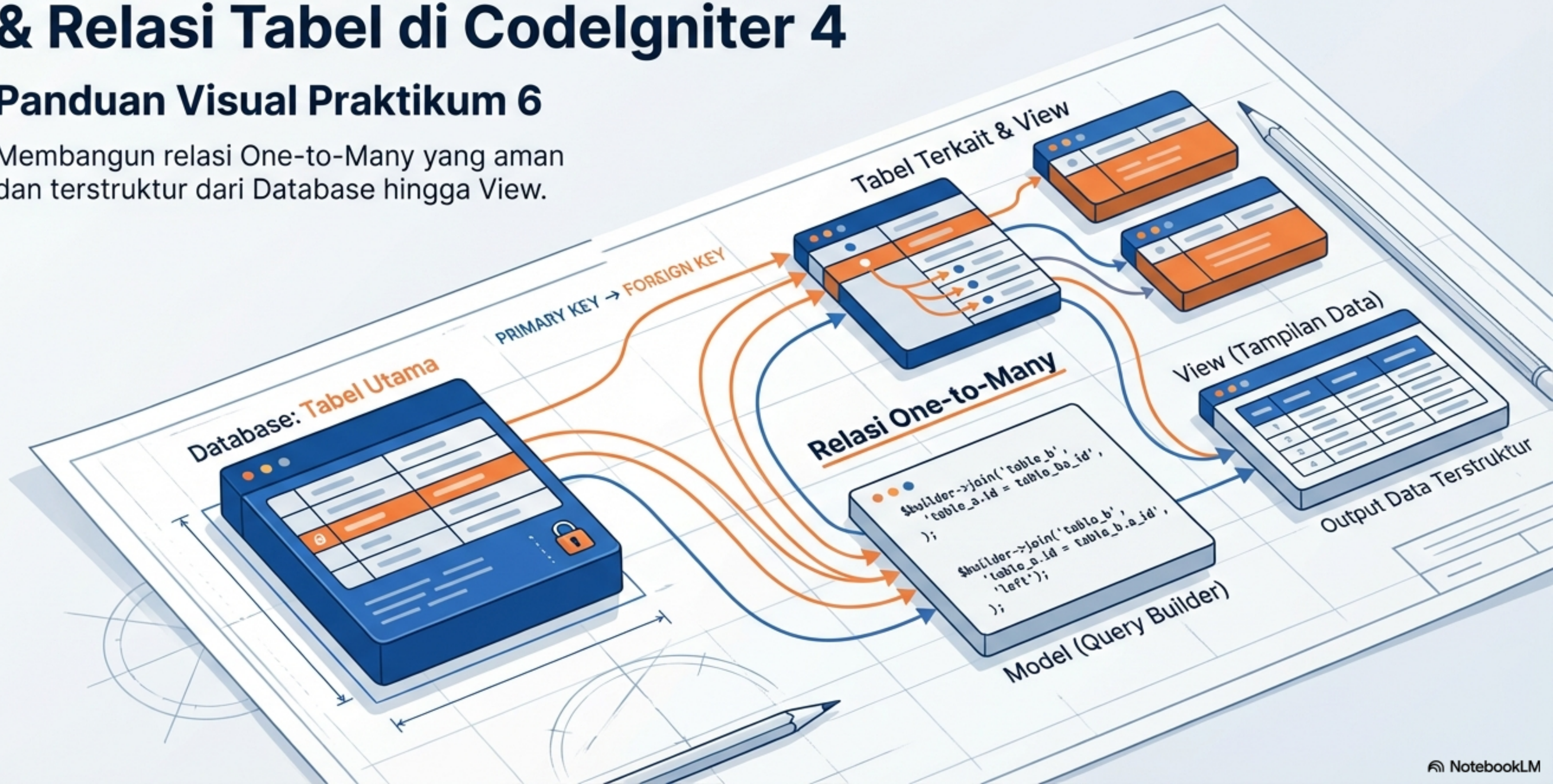


Arsitektur Data Berelasi: Menguasai Query Builder & Relasi Tabel di CodeIgniter 4

Panduan Visual Praktikum 6

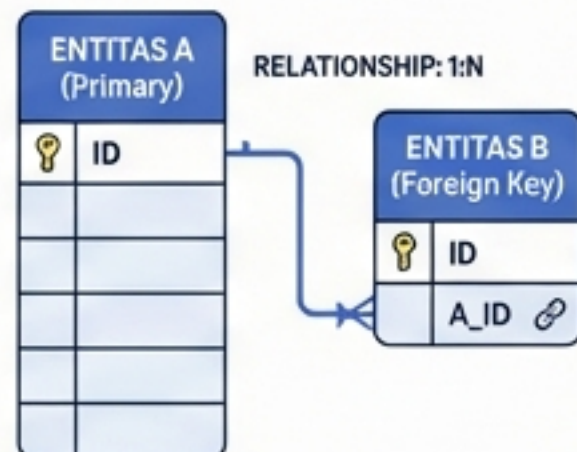
Membangun relasi One-to-Many yang aman dan terstruktur dari Database hingga View.



Tiga Objektif Utama Pembangunan Sistem Hari Ini

1. Relasi Basis Data

Mengonfigurasi arsitektur One-to-Many antar entitas tabel secara presisi.



2. Manipulasi Query Builder

Menghindari penulisan SQL manual yang rentan dengan sintaks method chaining CI4 yang aman.

SQL Manual (Vulnerable)

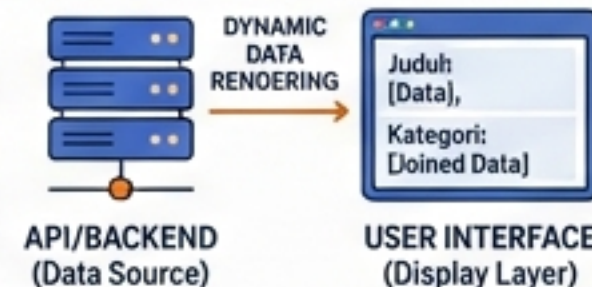
```
SELECT * FROM tables  
WHERE id = $input;
```

CI4 Method Chaining (Safe)

```
$builder->select('*')->  
where('id', $input)->get();
```

3. Integrasi Antarmuka

Menarik dan merender silang data relasional ke dalam layar pengguna secara dinamis.



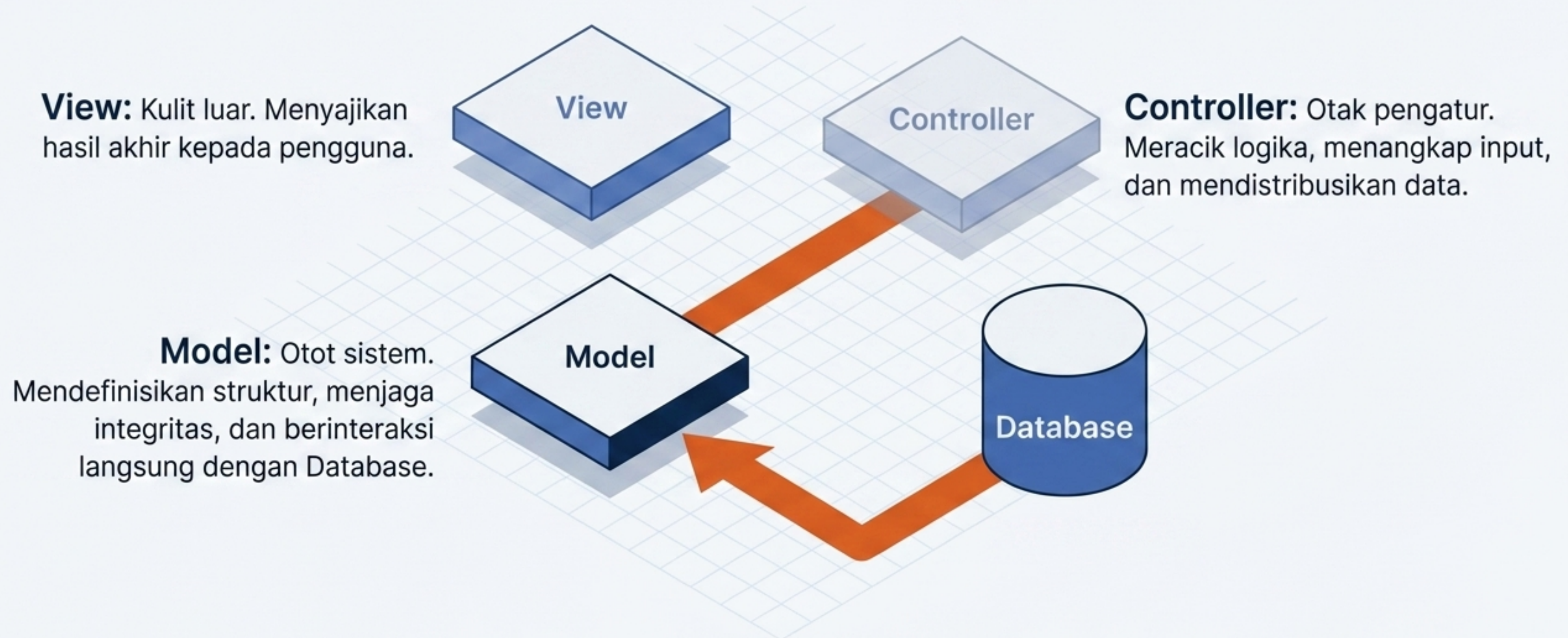
localhost/sistem/data

ID	Judul	Kategori	Status	Aksi
101	Proyek A - Pengembangan Modul	Software Development	Aktif	
102	Laporan Kinerja Q3	Business Intelligence	Selesai	
103	Audit Keamanan Sistem	Cybersecurity	Pending	
104	Migrasi Cloud Server	Infrastructure	Proses	

JOINED DATA RESULT (Relational Integration)

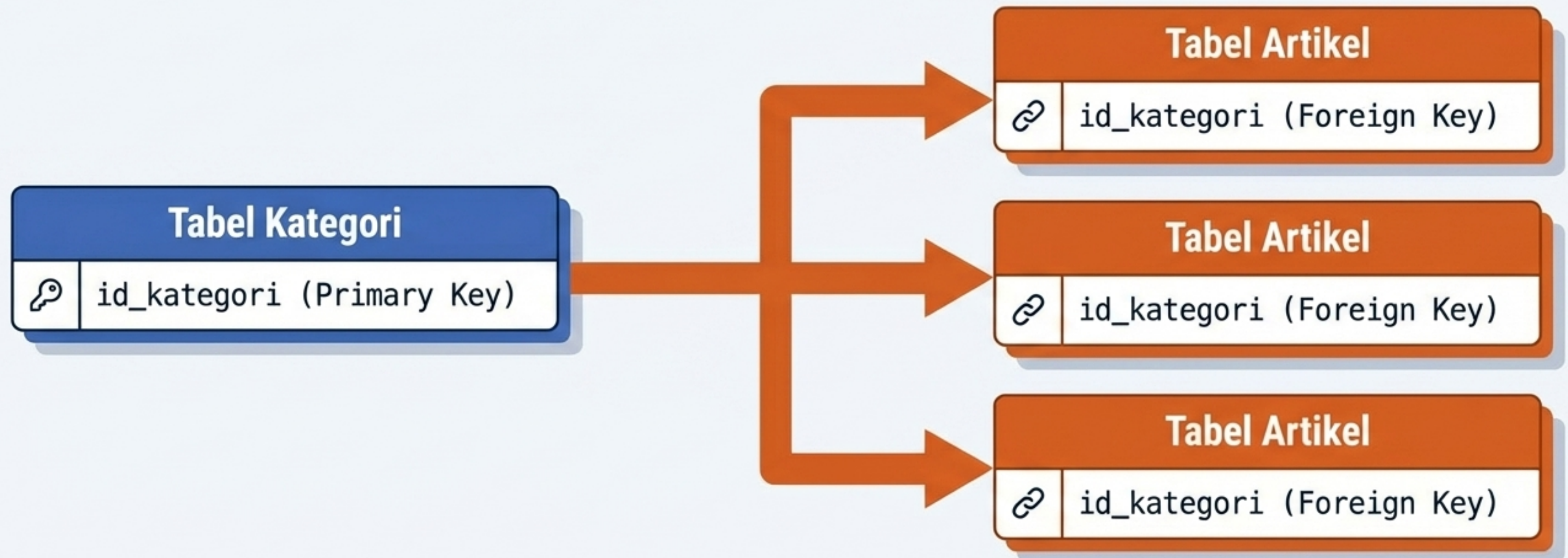
Segitiga Arsitektur MVC dan Posisi Krusial Relasi Data

Fokus kita dimulai pada penyatuan Model dan Database.



Anatomi Relasi Entitas One-to-Many

Satu payung, banyak turunan. Satu entitas Kategori dapat menaungi puluhan hingga ribuan Artikel. Kunci penghubungnya adalah menyematkan Foreign Key (`id_kategori`) pada tabel anak.



Evolusi Sintaks: Mengapa Beralih ke Query Builder?

Meninggalkan penulisan query manual yang berulang. Query Builder menawarkan sintaks cerdas berbasis objek yang secara otomatis mengamankan eksekusi data dan menyederhanakan manipulasi database kompleks.

Raw SQL

```
SELECT * FROM artikel JOIN kategori  
ON artikel.id_kategori =  
kategori.id_kategori
```



Rentan Injeksi



Sulit Dibaca

CI4 Query Builder

```
$builder->select(...)->join(...)
```



Aman (Escaped)



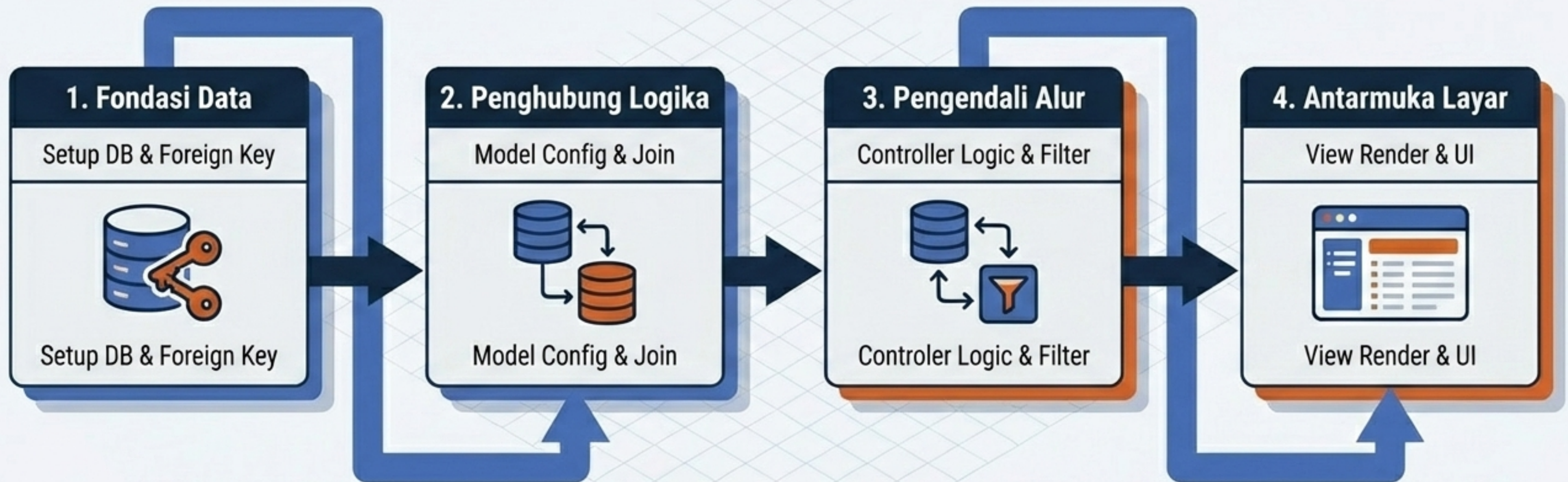
Terstruktur



Elegan

Peta Jalan Konstruksi Sistem Hari Ini

Empat fase berurutan untuk mentransformasi rancangan basis data mentah menjadi antarmuka web relasional yang interaktif.



Fase 1: Membangun Fondasi dan Mengunci Relasi Database

Menyiapkan rumah untuk entitas baru. Penambahan Foreign Key pada tabel artikel bukan sekadar formalitas—ini adalah pengunci (constraint) yang memaksa integritas data relasional di level server.

```
CREATE TABLE kategori (  
  id_kategori INT(11) AUTO_INCREMENT,  
  nama_kategori VARCHAR(100) NOT NULL,  
  slug_kategori VARCHAR(100),  
  PRIMARY KEY (id_kategori)  
);
```

```
ALTER TABLE artikel  
  ADD COLUMN id_kategori INT(11),  
  ADD CONSTRAINT fk_kategori_artikel  
    FOREIGN KEY (id_kategori) REFERENCES kategori(id_kategori);
```


Fase 2: Mendaftarkan Parameter Entitas pada Model

Framework CI4 membutuhkan izin eksplisit. Jika `id_kategori` tidak ditambahkan ke dalam **\$allowedFields** pada **ArtikelModel**, sistem akan secara otomatis memblokir setiap upaya penyimpanan relasi tersebut.

KategoriModel.php

```
protected $allowedFields =  
    ['nama_kategori',  
     'slug_kategori']  
];
```

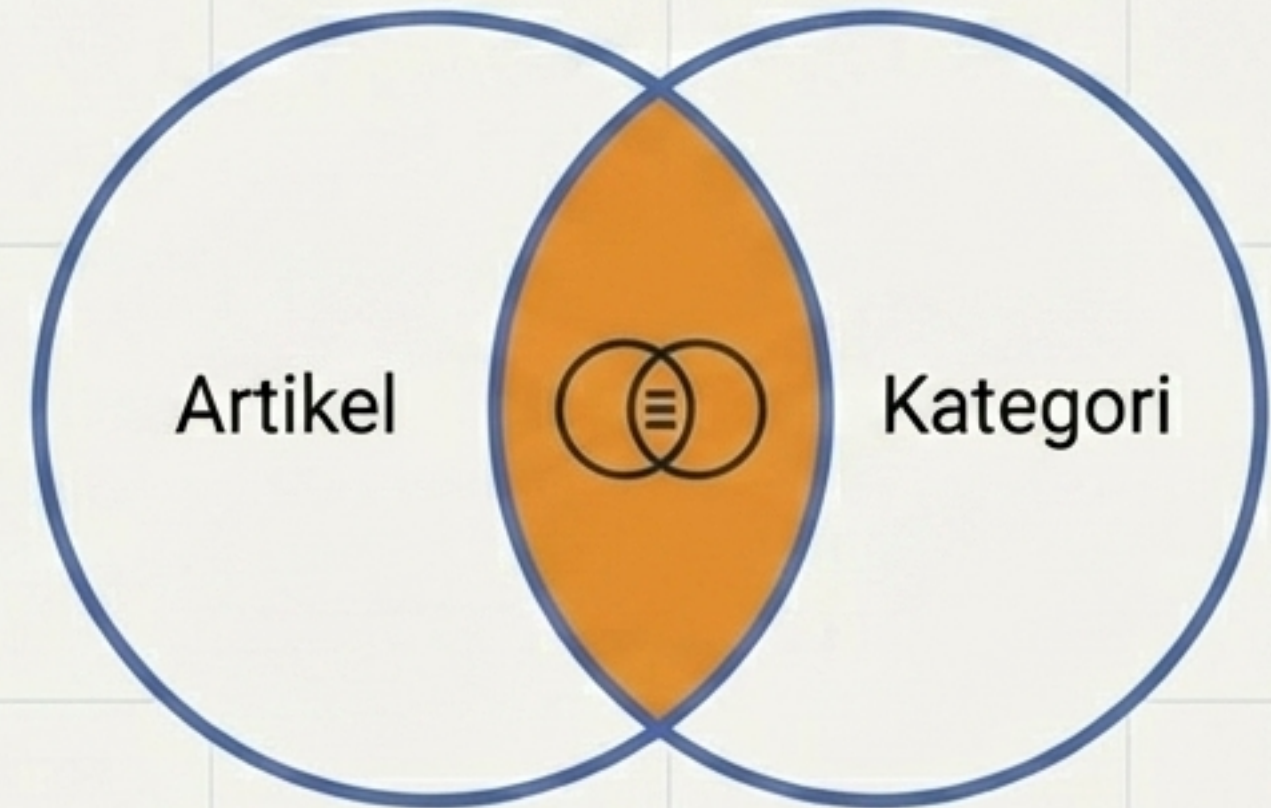
ArtikelModel.php

```
protected $allowedFields = [  
    'judul',  
    'isi',  
    'status',  
    'slug',  
    'gambar',  
    'id_kategori']  
];
```


Bedah Sintaks: Kekuatan Integrasi JOIN

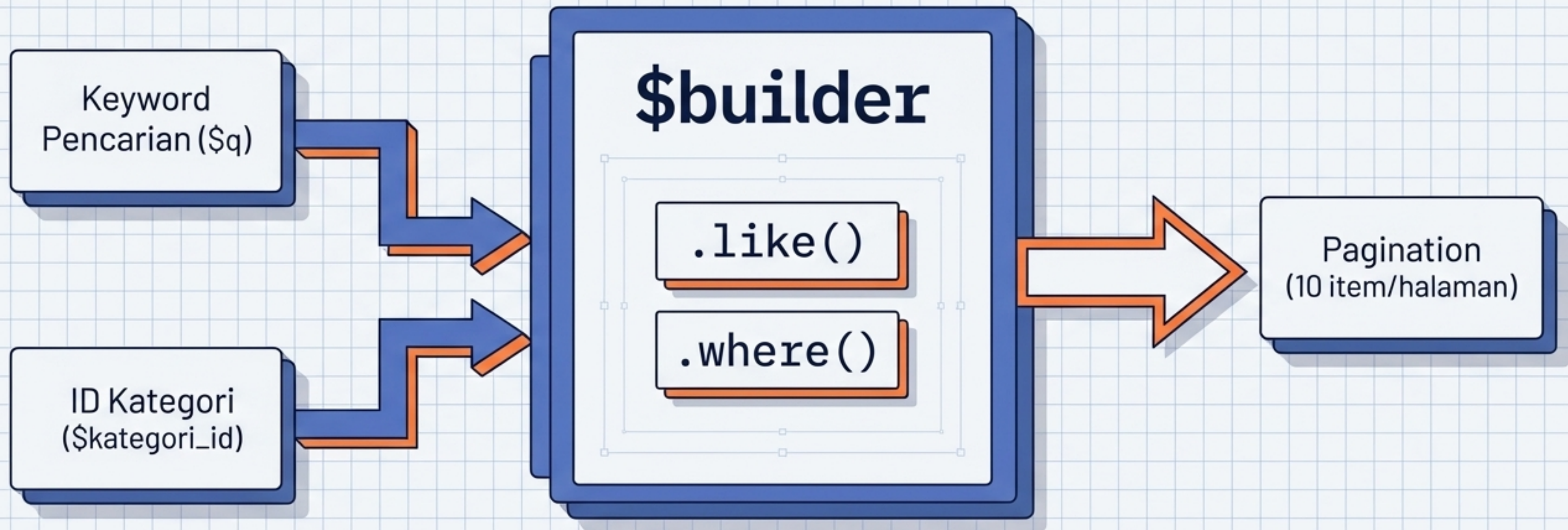
Di sinilah pertukaran data terjadi. Method **getArtikelDenganKategori()** menarik seluruh entitas artikel sekaligus mengikat nama kategori dari tabel terpisah dalam satu eksekusi tunggal.

```
->join('kategori',  
      'kategori.id_kategori =  
      artikel.id_kategori')
```



Fase 3: Controller Sebagai Dirigen Logika Alur Data

Controller meracik permintaan dinamis. Objek \$builder bertugas menumpuk filter pencarian, filter kategori, dan memecah halaman (pagination) sebelum merendernya secara utuh ke sistem antarmuka.



Mengamankan Pintu Masuk Data dengan Validasi

Mencegah anomali data (**Orphaned Data**). Logika validasi pada fungsi insert dan update memastikan bahwa setiap artikel baru harus secara mutlak ditautkan ke ID Kategori yang sah berwujud angka (integer).

```
'judul' => 'required',  
'id_kategori' => 'required|integer'
```

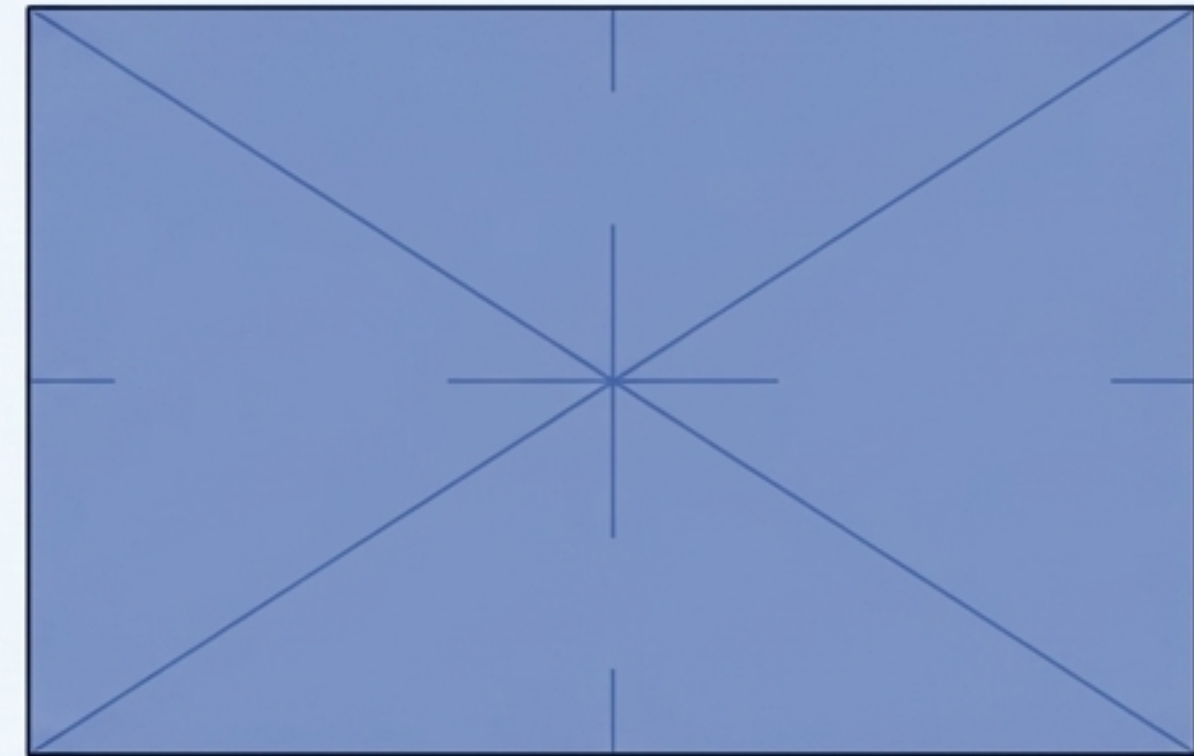


Fase 4: Mengekstraksi Relasi ke Layar Pengguna

Siklus selesai. Data yang diikat melalui struktur Model JOIN kini dapat dipanggil dengan sintaks sederhana di dalam View, memanifestasikan relasi backend menjadi informasi visual yang bermakna bagi pengguna.

index.php

```
<?html>
<head>
<dira ptaren execut="UTF8">
  <div class== "$blog_post, cert">
    <p>
      <p>Kategori: <?= $row['nama_kategori'] ?></p>
    </p>
  </div>
</body>
</html>
...
?>
```



Kategori: **Berita Teknologi**

Membangun Pilihan Interaktif via View Formulir

Formulir yang hidup. Elemen dropdown tidak ditulis secara manual, melainkan dilooping secara dinamis. Setiap kali admin menambahkan kategori baru di pangkalan data, opsi formulir akan langsung menyesuaikan.

```
<select name='id_kategori'>
  <?php foreach($kategori as $k): ?>
    <option
      value='<?= $k['id_kategori']; ?>'
      ><?= $k['nama_kategori']; ?>
    </option>
  <?php endforeach; ?>
</select>
```

Kategori



^	
Berita Teknologi	value='1'
Artikel Kesehatan	value='2'
Tutorial Pemrograman	value='3'
Review Gadget	value='4'

Verifikasi Kualitas Eksekusi CRUD

Kode yang selesai diketik belum tentu kode yang siap rilis. Uji coba menyeluruh memastikan bahwa empat fungsi esensial (CRUD) mampu mempertahankan integritas relasional tanpa menghasilkan error.



Tampil Daftar Artikel

Menampilkan nama relasi pada tabel utama.



Tambah Artikel Baru

Sistem dropdown kategori valid dan menyimpan ID.



Ubah Entitas

Update ID relasional ke kategori baru sukses.



Hapus Data

Artikel terhapus aman tanpa sisa di relasi.

Misi Anda Selanjutnya: Dokumentasi & Ekspansi

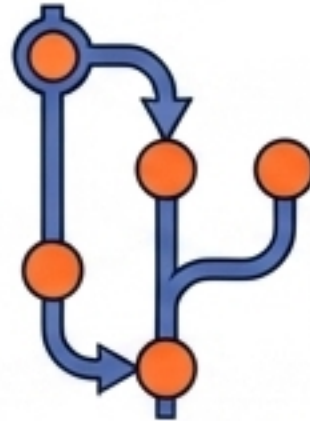
Struktur inti telah berdiri. Selesaikan tantangan berikutnya untuk menyempurnakan sistem dan rekam jejak digital profesional Anda.

LANGKAH 1



Modifikasi antarmuka `detail.php` untuk memasukkan nama kategori.

LANGKAH 2



Lakukan Commit rutin pada repositori GitHub Lab7Web.

LANGKAH 3



Tuliskan log perubahan ke dalam `README.md` beserta tangkapan layar (screenshot).

